- CPSC 375: High-Performance Computing

CPSC 375: High-Performance Computing

Spring 2026

- Syllabus
- Schedule
- Upload
- The Cluster Project

Getting Started with OpenMP Programming

This lab introduces OpenMP, a library for parallel programming in C. Through a series of small programs, you will create parallel regions, run loops across multiple threads, observe race conditions, and learn how synchronization and data-sharing clauses ensure correct results.

Exercise 1: First OpenMP Program

```
#include <stdio.h>
#include <omp.h>

int main() {
   #pragma omp parallel
   {
     printf("Hello from thread %d\n", omp_get_thread_num());
   }
}
```

Compile

```
$ gcc -fopenmp -o hello_openmp hello_openmp.c
```

Run

```
$ ./hello_openmp
```

Questions

- How many lines are printed?
- Does the order change when you run again?
- What is the smallest thread ID?

Exercise 2: Controlling Number of Threads

Modify the program

```
#include <stdio.h>
#include <omp.h>

int main() {
   omp_set_num_threads(4);

   #pragma omp parallel
   {
      printf("Thread %d running\n", omp_get_thread_num());
```

```
    }
}
```

Compile and run again

```
$ gcc -fopenmp -o hello_openmp hello_openmp.c
$ ./hello_openmp
```

Experiment

- Change 4 to 2
- Change 4 to 8

What happens to the output?

Exercise 3: Parallel Loop

Sequential version

```
#include <stdio.h>

int main() {
   for (int i=0; i<8; i++)
      printf("i = %d\n", i);
}
```

Parallel version

```
#include <stdio.h>
#include <omp.h>

int main() {
   #pragma omp parallel for
   for (int i=0; i<8; i++)
   {
      printf("Thread %d handles i=%d\n", omp_get_thread_num(), i);
   }
}
```

Questions

- Are loop iterations printed in order?
- Which thread handles each iteration?
- Does the output change each run?

Exercise 4: Race Condition

Program: race.c

```
#include <stdio.h>
#include <omp.h>

int main() {
   int sum = 0;

   #pragma omp parallel for
   for (int i=0; i<1000; i++)
   {
      sum += 1;
   }
```

```
   printf("Sum = %d\n", sum);
}
```

Compile and run

```
$ gcc -fopenmp -o race race.c
$ ./race
```

Expected result

```
Sum = 1000
```

You may see different results each run.

Question

Why does this happen?

Exercise 5: Fix with critical

Correct program: critical.c

```
#include <stdio.h>
#include <omp.h>

int main() {
   int sum = 0;

   #pragma omp parallel for
   for (int i=0; i<1000; i++)
   {
      #pragma omp critical
     {
       sum += 1;
     }
   }
   printf("Sum = %d\n", sum);
}
```

Compile and run

```
$ gcc -fopenmp -o critical critical.c
$ ./critical
```

Result

```
Sum = 1000
```

Now the program works correctly.

Exercise 6: Fix with Reduction

Correct program: reduction.c

```
#include <stdio.h>
#include <omp.h>

int main() {
   int sum = 0;
```

```
    #pragma omp parallel for reduction(+:sum)
    for (int i=0; i<1000; i++)
    {
       sum += 1;
    }
    printf("Sum = %d\n", sum);
}
```

Here, a `reduction` combines private copies of a variable from multiple threads into a single final value using a specified operation (such as +, *, max, or min) at the end of a parallel region.

Compile and run

```
$ gcc -fopenmp -o reduction reduction.c
$ ./reduction
```

Result

```
Sum = 1000
```

Now the program works correctly.

Exercise 7: Shared Variable Behavior

Program: shared_example.c

```
#include <stdio.h>
#include <omp.h>

int main() {
   int x = 5;

   #pragma omp parallel
   {
      x = x + 1;
      printf("Thread %d: x = %d\n", omp_get_thread_num(), x);
   }
   printf("Final x = %d\n", x);
}
```

Compile and run:

```
$ gcc -fopenmp -o shared_example shared_example.c
$ ./shared_example
```

Questions

- Do all threads print the same value?
- What is the final value of x?
- Why might the output change each run?

Exercise 8: Using private

Program: private_example.c

```
#include <stdio.h>
#include <omp.h>

int main() {
   int x = 5;
```

```
   #pragma omp parallel private(x)
   {
      x = x + 1;
      printf("Thread %d: x = %d\n", omp_get_thread_num(), x);
   }
printf("Final x = %d\n", x);
}
```

Compile and run

```
$ gcc -fopenmp -o private_example private_example.c
$ ./private_example
```

Questions

- What value does each thread print?
- Why might the values appear unpredictable?
- What is the final value of x in main?

Exercise 9: Using firstprivate

Program: firstprivate_example.c

```
#include <stdio.h>
#include <omp.h>

int main() {
   int x = 5;

   #pragma omp parallel firstprivate(x)
   {
      x = x + 1;
      printf("Thread %d: x = %d\n", omp_get_thread_num(), x);
   }
   printf("Final x = %d\n", x);
}
```

Here, with `firstprivate`, each thread gets its own copy initialized with the original value.

Compile and run

```
$ gcc -fopenmp -o firstprivate_example firstprivate_example.c
$ ./firstprivate_example
```

Questions

- What value does each thread print?
- Is `x` initialized for each thread?
- Does the value of `x` in main change?

Exercise 10: Using lastprivate

Program: lastprivate_example.c

```
#include <stdio.h>
#include <omp.h>

int main() {
   int x = 0;
```

```
    #pragma omp parallel for lastprivate(x)
    for (int i = 0; i < 8; i++)
    {
       x = i;
       printf("Thread %d: i=%d x=%d\n", omp_get_thread_num(), i, x);
    }
    printf("Final x = %d\n", x);
}
```

Here, with `lastprivate`, the last iteration's value is copied back to the original variable.

Compile and run

```
$ gcc -fopenmp -o lastprivate_example lastprivate_example.c
$ ./lastprivate_example
```

Questions

- Which value is printed as the final `x`?
- Why is it not the value from every thread?
- What does `lastprivate` guarantee?

Exercise 11: Combine firstprivate and lastprivate

Modify the loop:

```
#pragma omp parallel for firstprivate(x) lastprivate(x)
```

Run the program again and observe the behavior.

Questions

- What value does each thread start with?
- What is the final value after the loop?

Exercise 12: Parallel Summation of Array

Create an array:

```
int A[1000];
```

Initialize:

```
A[i] = i;
```

Write a program that computes the sum of an array using OpenMP and reduction.